RETRIEVE · REASON · ACT

RAG & AI Agent

LLM을 '더 정확하게', 그리고 '직접 일하게' 만드는 방법

KUGGLE LLM 세션

AGENDA

오늘 다룰 내용

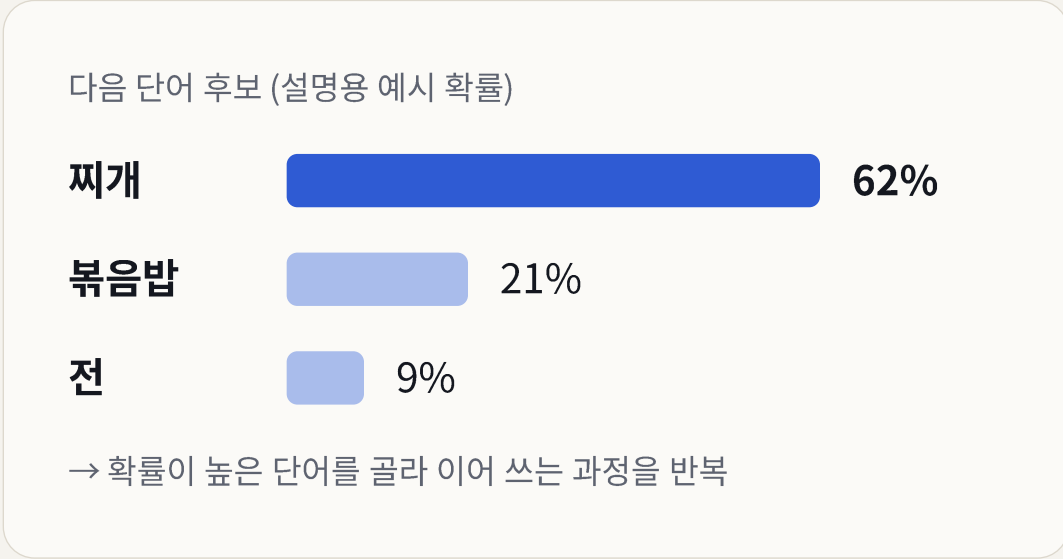
01	**LLM의 한계**	왜 LLM 혼자서는 부족할까?
02	**RAG**	필요한 자료를 찾아보고 답하기
03	**AI Agent**	도구를 써서 스스로 일하기
04	**정리**	둘의 차이와 함께 쓰는 방법

LLM은 ‘다음 단어 예측기’

오늘 점심은 김치 _____

방대한 글을 학습해서, 문맥상 가장 그럴듯한 다음 단어를 하나씩 이어 붙이며 문장을 만듭니다.

그래서 ‘그럴듯한 답’과 ‘정확한 답’은 다를 수 있습니다.



LLM 혼자서는 부족한 네 가지 이유

01

지식 마감

학습한 시점 이후의 정보는 알지 못합니다

→ RAG로 해결

02

환각

모르는 내용도 그럴듯하게 지어냅니다

→ RAG로 완화

03

내 데이터 모름

사내 문서나 개인 파일은 학습한 적이 없습니다

→ RAG로 해결

04

행동 불가

답을 '말'할 뿐, 직접 실행하지는 못합니다

→ Agent로 해결

이 한계를 보완하는 대표적인 방법: RAG와 AI Agent

PART 02

RAG

Retrieval-Augmented Generation · 검색 증강 생성

질문과 관련된 자료를 먼저 찾아보고, 그 자료를 근거로 답하게 만드는 방법

RAG vs 파인튜닝: 언제 무엇을 쓸까

RAG

필요한 자료를 찾아서 참고

모델	그대로 둠
지식 추가	문서만 교체하면 바로 반영
근거 제시	참고한 출처를 보여줄 수 있음

이럴 때 적합

자주 바뀌는 정보, 사내 문서 Q&A, 출처가 중요한 답변

FINE-TUNING

추가 학습으로 모델 자체를 조정

모델	가중치를 다시 학습
지식 추가	데이터 준비 + 재학습 필요
근거 제시	출처를 따로 보여주기 어려움

이럴 때 적합

일정한 말투·출력 형식, 특정 분야의 용어와 작업 방식

02 · RAG — 동작 흐름

RAG 기반 응답 생성 과정

STEP 1 · QUERY

질문

사용자가 질문을 입력

→

STEP 2 · RETRIEVE

검색

벡터 DB에서 관련 문서 조각 찾기

→

STEP 3 · AUGMENT

증강

질문 + 찾은 문서로 프롬프트 구성

→

STEP 4 · GENERATE

생성

문서를 근거로 LLM이 답변

예시

“우리 학회 회비 얼마야?” → 회칙 문서에서 ‘회비’ 조항 검색 → 조항 + 질문을 LLM에 전달 → “회칙 제5조에 따르면 …”

RAG의 문서 인덱싱 과정

STEP 1 · LOAD

수집

PDF, 웹페이지, DB 등 문서 모으기

→

STEP 2 · CHUNK

청킹

긴 문서를 작은 조각으로 자르기

→

STEP 3 · EMBED

임베딩

각 조각을 숫자 벡터로 바꾸기

→

STEP 4 · STORE

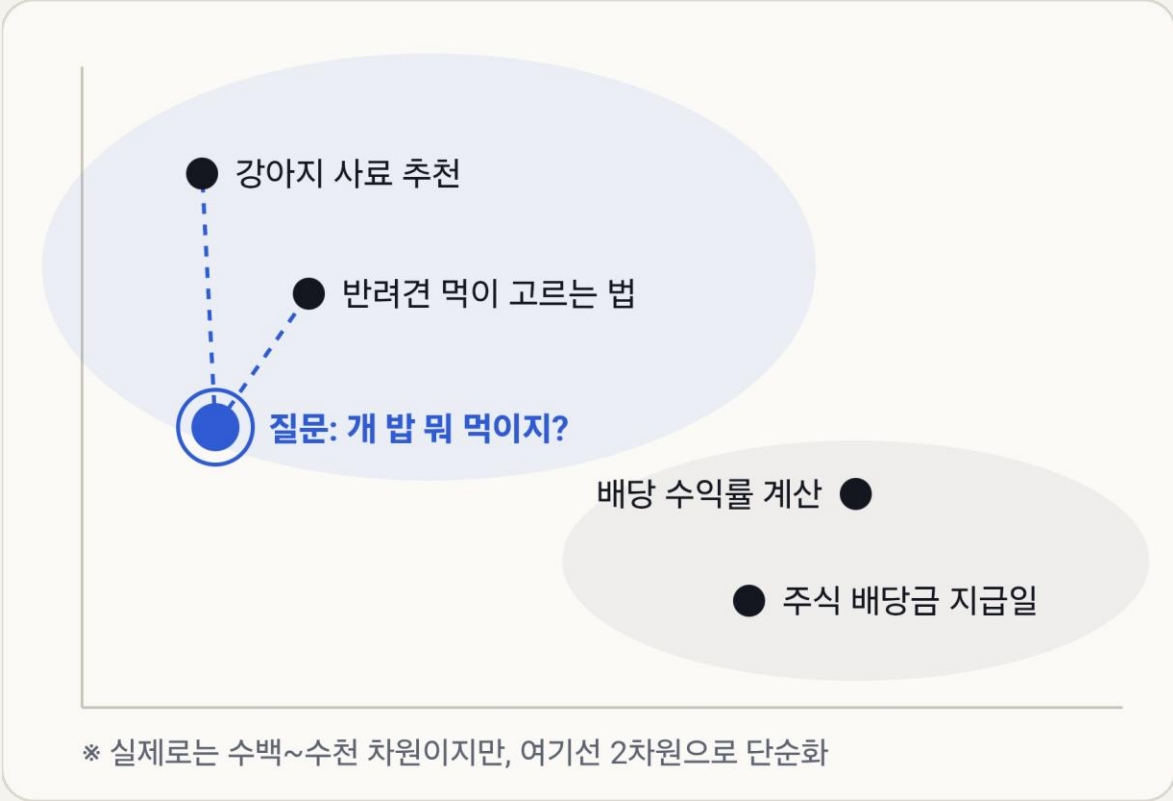
저장

벡터 DB에 보관 (예: FAISS, Chroma)

포인트

이 준비는 미리 한 번만 해 두고, 질문이 들어올 때마다 앞 장의 '검색' 단계만 빠르게 수행합니다.

임베딩: 문장의 '의미'를 좌표로 바꾸기



의미가 비슷하면 가까이

문장을 숫자 벡터로 바꾸면, 뜻이 비슷한 문장끼리 가까운 곳에 모입니다.

단어가 달라도 찾는다

'강아지'와 '반려견'처럼 표현이 달라도 의미로 검색할 수 있습니다.

거리는 숫자로 계산

보통 코사인 유사도로 질문과 가장 가까운 문서 조각을 고릅니다.

LLM 프롬프트 예시

[지시] 아래 참고 문서만 근거로 답하세요. 문서에 없으면 “모르겠습니다”라고 답하세요.

[참고 문서 1] OO학회 회칙 제5조 — 회비는 학기당 3만 원으로 한다.

[참고 문서 2] 2학기 공지 — 회비 납부 기한은 9월 30일까지.

[질문] 이번 학기 회비 얼마고, 언제까지 내야 해?

핵심 **모델을 다시 학습시키는 게 아니라, 검색으로 찾은 자료를 ‘프롬프트에 붙여 주는 것’**

RAG의 장점과 주의할 점

장점

최신 정보 반영
문서만 바꾸면 바로 반영됩니다

근거·출처 제시
어떤 문서를 참고했는지 보여줄 수 있습니다

재학습 불필요
파인튜닝 없이 새 지식을 추가합니다

주의할 점

검색 품질 = 답변 품질
엉뚱한 문서를 찾으면 답도 틀립니다

설정에 민감
청크 크기, 가져올 문서 수에 따라 달라집니다

환각이 0이 되지는 않음
문서를 잘못 해석할 가능성은 남습니다

PART 03

AI Agent

생각하고, 도구를 쓰고, 결과를 보고, 다시 생각한다

LLM이 스스로 계획을 세우고 도구를 사용해서, 주어진 목표를 끝까지 수행하는 시스템

03 · AI AGENT

챗봇과 에이전트는 무엇이 다를까

구분	챗봇 (일반 LLM)	AI Agent
하는 일	질문에 답변한다	목표를 받아 작업을 끝낸다
동작 방식	한 번 묻고, 한 번 답한다	생각 → 행동 → 관찰을 반복한다
외부 도구	기본적으로 사용하지 않는다	검색·코드 실행·API를 직접 호출한다
요청 예시	“이 데이터 분석하는 방법 알려줘”	“이 데이터 분석해서 보고서까지 만들어줘”

챗봇은 ‘조언자’, 에이전트는 ‘실행자’에 가깝습니다

AI Agent를 이루는 네 가지

BRAIN

두뇌 (LLM)

상황을 이해하고 다음에 할 행동을 결정

예: GPT, Claude, Gemini

TOOLS

도구

LLM이 직접 못 하는 일을 대신 수행

예: 웹 검색, 코드 실행, DB 조회, API

MEMORY

메모리

지금까지의 대화와 작업 결과를 기억

예: 대화 기록, 중간 결과 저장

PLANNING

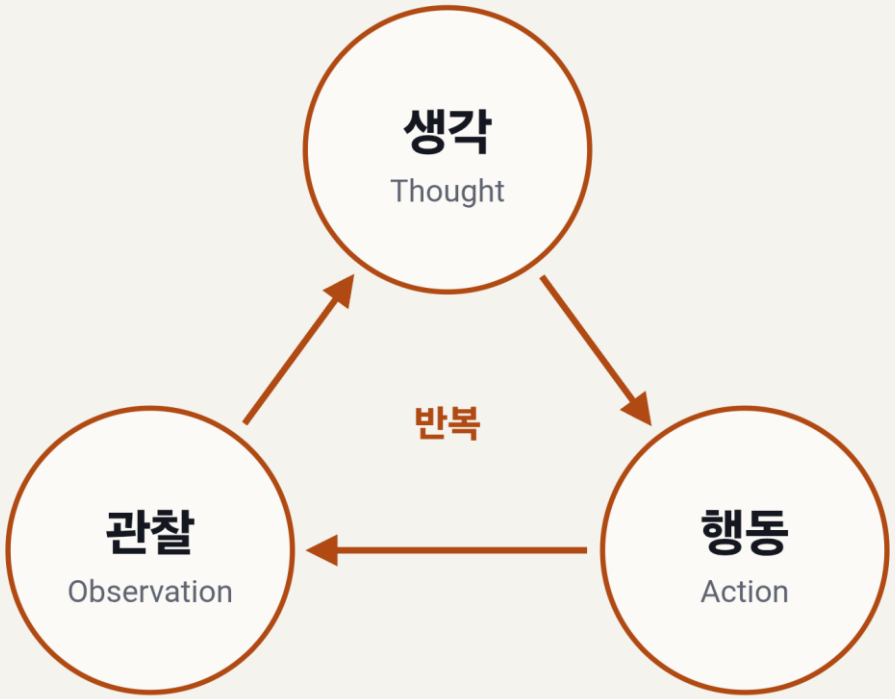
계획

큰 목표를 작은 단계로 나눔

예: 분석 → 불러오기·정제·시각화

LLM 하나가 아니라, LLM을 중심으로 도구·기억·계획을 묶은 '시스템'

AI 에이전트의 의사결정 구조



생각 “지금 뭐가 필요하지?” 다음 행동을 판단

행동 알맞은 도구를 골라 실행 (검색, 코드 실행 등)

관찰 실행 결과를 확인하고 다시 ‘생각’으로

종료 **목표를 이루면 루프를 멈추고 최종 답변**

이 패턴을 ReAct(Reasoning + Acting)라고 부릅니다 · Yao et al., 2022

멀티 에이전트: 역할을 나눠 협업하기

ORCHESTRATOR

매니저 에이전트

"매출 이상치 원인 보고서" → 작업을 나눠 맡기고 결과를 합침

↓ 작업 분배 · ↑ 결과 보고

WORKER 1

수집 담당

SQL로 일별 매출 조회

WORKER 2

분석 담당

Python으로 이상치 탐색

WORKER 3

리서치 담당

사내 문서 검색 (RAG)

실제 활용 사례

Claude 리서치 Anthropic · 2025

리드 에이전트가 하위 에이전트들에게 검색을 나눠 맡겨 동시에 조사

ChatDev 연구 프로젝트 · 2023

CEO·CTO·프로그래머·테스터 에이전트가 대화하며 소프트웨어 개발

MetaGPT 오픈소스 · 2023

회사 업무 절차(SOP)를 본떠 기획자·설계자·엔지니어가 작업 분담

03 · AI AGENT

AI Agent 활용 시 고려사항

끝나지 않는 루프

같은 행동을 반복하며 시간·비용을 낭비

→ 최대 반복 횟수를 제한

비용과 속도

LLM을 여러 번 호출해서 느리고 비쌈

→ 단순 질문은 일반 LLM으로 충분

위험한 실행

삭제·결제·메일 발송처럼 되돌리기 힘든 행동

→ 중요한 단계는 사람이 최종 승인

오류 누적

앞 단계의 작은 실수가 끝까지 이어짐

→ 중간 과정을 로그로 확인

한 장 요약

LLM

똑똑하지만, 최신 정보와 내 데이터를 모르고 직접 행동할 수 없다

RAG

필요한 자료를 검색해 붙여 주어 ‘근거 있는 답’을 만든다

AI AGENT

생각 → 행동 → 관찰을 반복하며 도구로 ‘일을 끝낸다’

AI AGENT의 도구 상자

웹 검색	코드 실행
계산기	**문서 검색 (RAG)**

필요하다고 판단할 때 에이전트가 직접 검색을 호출
→ **Agentic RAG**

THANK YOU

Q & A

들어주셔서 감사합니다

참고 · Lewis et al. (2020), Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks
참고 · Yao et al. (2022), ReAct: Synergizing Reasoning and Acting in Language Models